

Vererbung und Polymorphie – Teil 2

Gemeinsame Verträge und Polymorphie

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 11. Dezember 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan  überlebt den ersten Kontakt  mit der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Vererbung und Polymorphie

Plan für diese Woche

 Super- und Unterklassen

 Vererbung entwerfen

 Pair Programming

 Ist-Ein oder Hat-Ein

 Sinnvoller Einsatz

 Wozu? – Nutzen von Vererbung

 Gemeinsames Protokoll

 Polymorphie

 Überschreiben und Überladen

 Pair Programming

 Spaß

📖 Vererbung und Polymorphie

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

🎯 Wie hilft Vererbung, doppelten Code zu vermeiden?

📜 Was haben Protokolle und Verträge mit Vererbung zu tun?

🎨 „Bei Polymorphie können Referenz- und Objekttyp verschieden sein.“ Was ist damit gemeint?

🔧 Was ist der Unterschied zwischen Überschreiben und Überladen von Methoden?

👤 Welche Fragen haben Sie zu diesem Kapitel?

7 Vererbung und Polymorphie

Besser leben in Objectville

Wir waren unterbezahlte und überarbeitete Programmierer, bis wir es mit dem Polymorphie-Plan versucht haben. Dank des Plans sehen wir einer rosigen Zukunft entgegen. Und das können Sie auch!



Planen Sie Ihre Programme mit der Zukunft im Blick. Was wäre es Ihnen wert, wenn es eine Möglichkeit gäbe, Java-Code so zu schreiben, dass Sie mehr Urlaub nehmen könnten? Was würde es Ihnen bedeuten, wenn Sie Code so schreiben könnten, dass *jemand anders* ihn **einfach** erweitern könnte? Und was, wenn Sie Code schreiben könnten, der schon auf diese lästigen Änderungen der Spezifikation (wie immer in letzter Minute) vorbereitet ist? Würde Sie das interessieren? Dann ist heute Ihr Glückstag. Für nur drei einfache Raten zu je 60 Minuten Ihrer Zeit kann all dies Ihnen gehören. Der Polymorphie-Plan zeigt Ihnen die fünf einfachen Schritten für besseres Klassendesign, die drei Tricks der Polymorphie, die acht Möglichkeiten, Code flexibel zu gestalten, und wenn Sie innerhalb der nächsten zehn Minuten anrufen, erhalten Sie außerdem eine Bonusstunde mit vier Tipps zur besseren Nutzung der Vererbung gratis dazu. Zögern Sie nicht! Dieses einmalige Angebot gibt Ihnen die Designfreiheit und Flexibilität in der Programmierung, die Sie verdienen. Es ist schnell. Es ist einfach. Und es steht ab sofort zur Verfügung. Fangen Sie noch heute damit an, und Sie erhalten eine extra Abstraktionsebene gratis dazu!



Wozu? – Nutzen von Vererbung

Duplizierter Code

🎯 Wozu? – Nutzen von Vererbung

👤 Wir werden doppelten Code los, indem wir das **allgemeine Verhalten** einer Gruppe von Klassen abstrahieren und es in eine abstraktere **Superklasse** packen.

🏆 Wir müssen diesen Code nur **an einer Stelle** im Programm ändern, nämlich in der Superklasse...

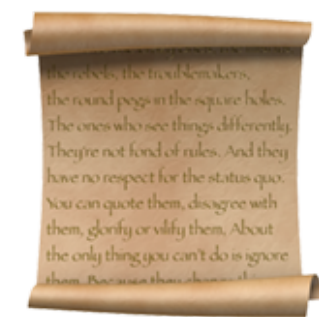
🪄 ... und die Unterklassen erhalten diese **Änderungen automatisch** – wie von „Zauberhand“.

Aha! Ich
kann leicht
flexiblen
Code
entwickeln,
der effizient
und sauber
ist.



 Welcher **eine** Gedanke zu
„ Wozu? – Nutzen von
Vererbung“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren



Gemeinsames Protokoll

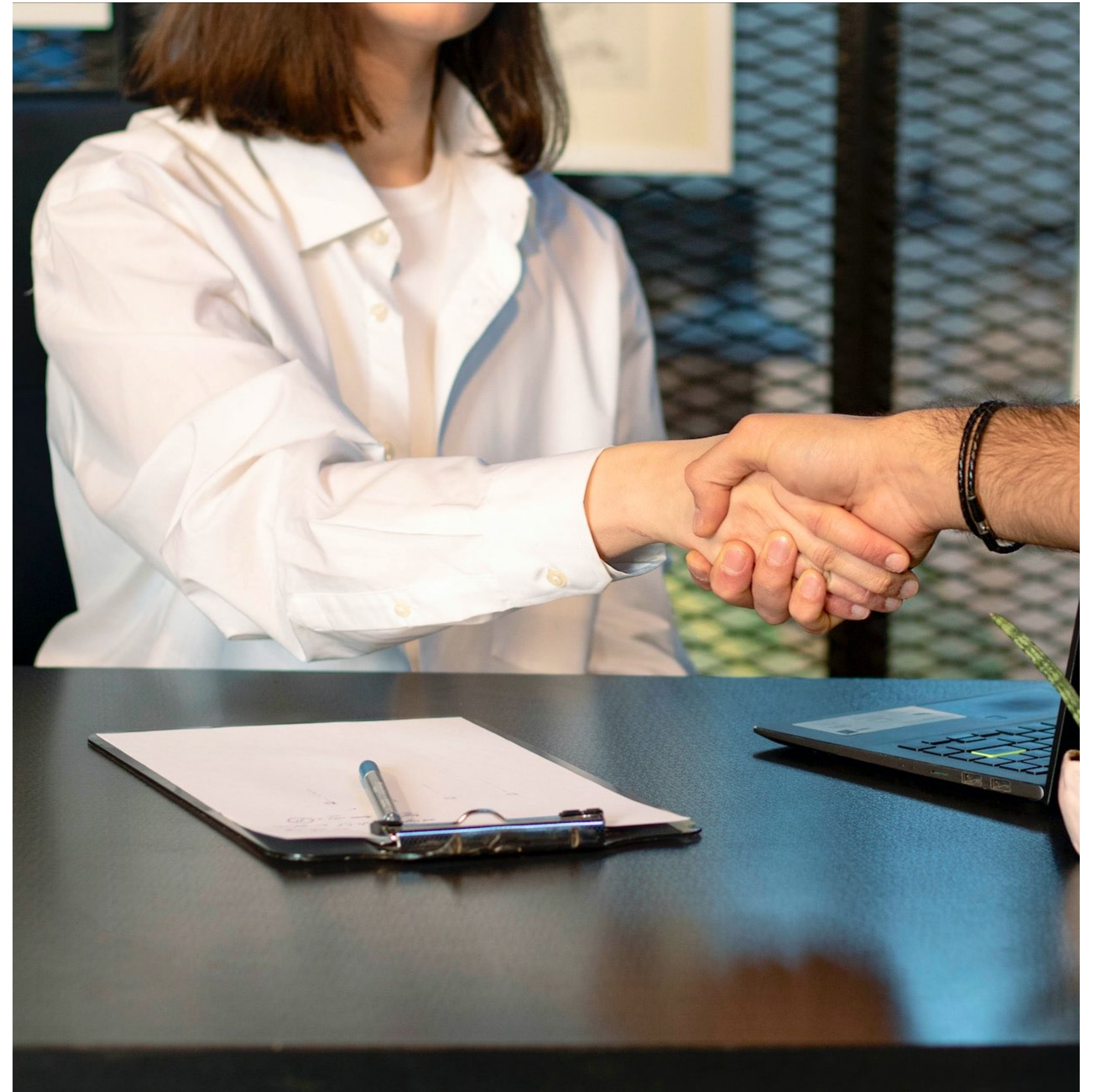
Sehr schön, abgemacht!

Gemeinsames Protokoll

 Durch Vererbung stellen wir sicher, dass alle Klassen, die unter einem bestimmten Supertyp gruppiert sind, **alle Methoden** haben, die der **Supertyp** hat.

 Wir definieren also durch Vererbung ein **gemeinsames Protokoll** für eine Gruppe verwandter Klassen.

 Ein Vertrag darüber, was **alle Unterklassen** an Verhalten mitbringen.



„Okay... Und was **konkret** bedeutet das?“

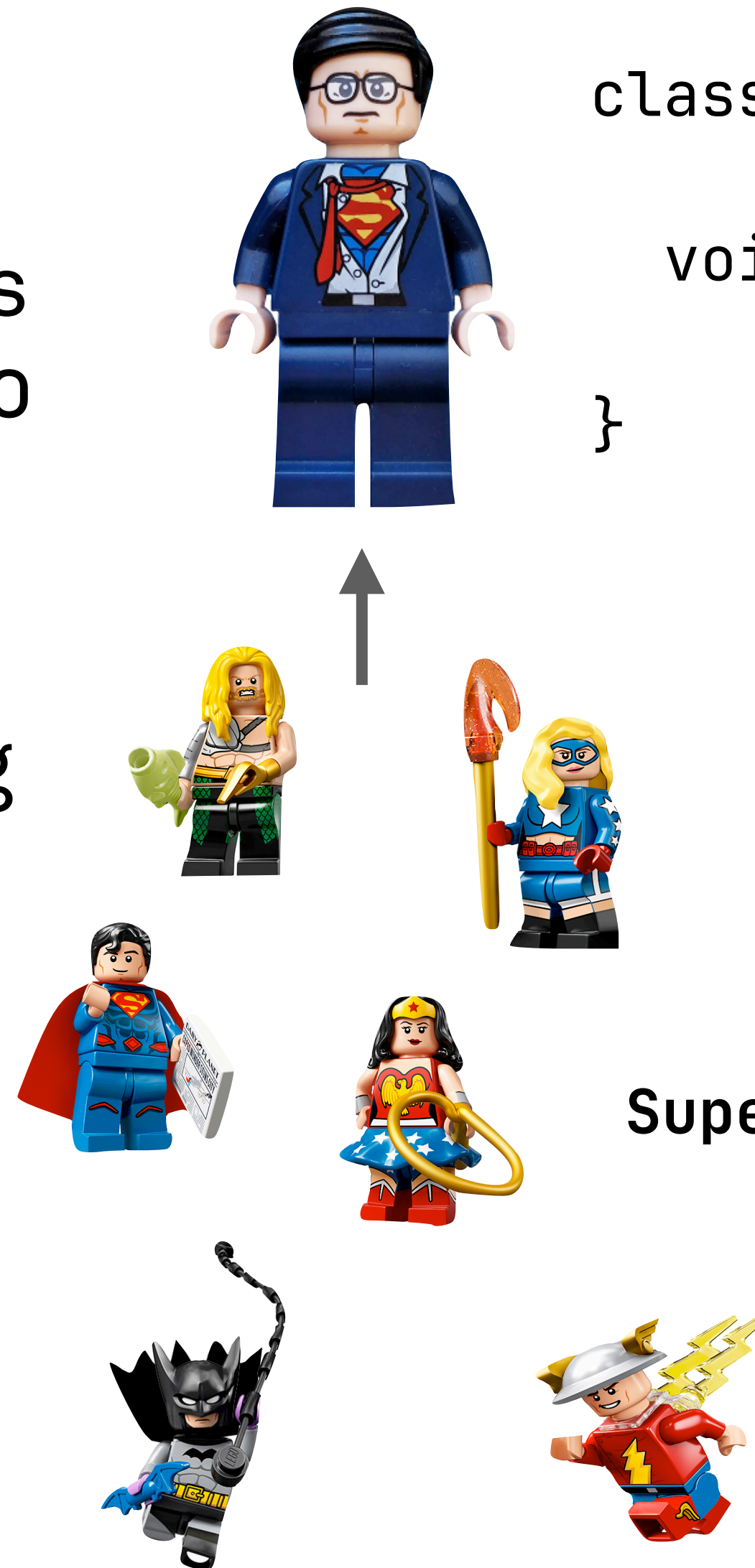
Noel – neugierige Student:in im 1. Semester



Möge die Macht mir Dir sein!

Gemeinsames Protokoll

- Mit Vererbung sichern wir zu, dass alle Klassen, die unter Superhero gruppiert sind, alle Methoden haben, die dieser Supertyp hat.
- So definieren wir durch Vererbung ein gemeinsames Protokoll für den Superhero-Typ.
- Einen Vertrag, dass alle Superheros Mut beweisen können.



```
class Superhero {
```

```
    void showCourage() { /* 💪 */ }
```

```
}
```

Aha! Ich kann
solch' flexiblen
Code auch sehr
leicht erweitern.

```
Superhero myHero = new WonderWoman();
```


! Quiz

Gemeinsames Protokoll

-  Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
-  Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
-  Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Durch Vererbung kann sichergestellt werden, dass alle Klassen eines bestimmten Supertyps alle öffentlichen Methoden dieses Supertyps verwenden können.

Richtig!

Wenn der Code einer geerbten Methode in einer Superklasse geändert wird, können Unterklassen diesen geänderten Code erhalten, ohne dass in den Unterklassen selbst Änderungen vorgenommen werden müssen.

Richtig!

Vererbung wird verwendet, um eine Art gemeinsames Protokoll oder, mit anderen Worten, einen Vertrag für eine Reihe verwandter Klassen zu definieren.

Richtig!

Wenn Sie einen Subtyp für eine Gruppe von Klassen definieren, kann jede Superklasse dieses Subtyps überall dort verwendet werden, wo der Subtyp erwartet wird.

Umgekehrt: Supertyp und Unterklasse

 Welcher **eine** Gedanke zu
„ Gemeinsames Protokoll“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren



Polymorphie




```
Dog myDog = new Dog();
```

1. 

3. 
2. 

Deklaration, Erstellung und Zuweisung von Objekten

Wie Polymorphie funktioniert in aller Kürze

Eine Typfrage: Referenz und Objekt

🎨 Polymorphie



1) 3) 2)
WonderWoman myHero = new WonderWoman();

1) Eine Referenzvariable deklarieren

3) Das Objekt mit der Referenz verknüpfen

2) Ein Objekt erzeugen

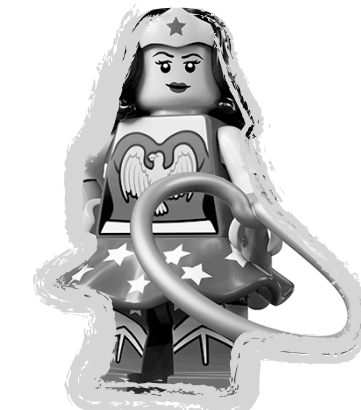
Okay, beide - Referenz und Objekt - haben also denselben Typ.

Eine Typfrage: Referenz vs. Objekt

Polymorphie



```
WonderWoman myHero = new WonderWoman();
```



Der Typ der Referenzvariablen ist als
Superhero deklariert...

... und das Objekt wird als neue
WonderWoman erzeugt.



```
Superhero myHero = new WonderWoman();
```

Besteht „Ist-Ein“-Test und
hält daher den Vertrag ein.



Aha! Mit Polymorphie können Referenz-
und Objekttyp unterschiedlich* sein.

*Solange der
Referenztyp eine
Superklasse des
Objektyps ist.

Polymorphie in Aktion

Ein starkes Team dank Polymorphie

🎨 Polymorphie

Ein Array des Typs Superhero deklarieren. Also ein Array, das Objekte vom Typ Superhero speichert.

```
Superhero[] team = new Superhero[3];
```

```
team[0] = new WonderWoman();
```

```
team[1] = new Batman();
```

```
team[2] = new Stargirl();
```

```
for (Superhero hero: team) {
```

```
    hero.showCourage();
```

```
}
```

Wir können jede beliebige Unterklasse von Superhero in das Superhero-Array stecken. Aufgrund „Ist-Ein“ und des Vertrags.

Wir durchlaufen das Array in einer Schleife...
... und rufen eine Methode der Klasse Superhero auf -
und jedes Objekt macht genau das, was es soll.



So beweisen - dank Polymorphie - erst Wonder Woman, dann Batman und zu guter Letzt Stargirl ihren unerschütterlichen Mut.

Polymorphe Argumente

Polymorphie

```
class BadGuy {  
    public void challenge(Superhero hero) {  
        // Den Helden am anderen Ende von ,hero'  
        // zu einem Kampf auffordern.  
        hero.showCourage();  
    }  
}
```

```
BadGuy bg = new BadGuy();
```

```
WonderWoman ww = new WonderWoman();  
Stargirl sg = new Stargirl();
```

```
bg.challenge(ww);  
bg.challenge(sg);
```



Der Parameter ,hero' kann
jeden beliebigen Superhero-Typ
als Argument nehmen.

Aha! Mit Polymorphie kann
ich Code schreiben, der bei
neuen Unterklassen nicht
geändert werden muss.

Wir erzeugen einen BadGuy
und Superheros als Argumente.

Die challenge()-Methode von BadGuy kann
jeden Superhero auffordern, den wir ihr
überreichen. Das funktioniert, solange die
Argumente eine Unterklasse von Superhero sind.

Die jeweilige showCourage()-Methode des
konkreten Superheros wird aufgerufen.

Polymorphe Rückgabewerte

🎨 Polymorphie

```
Superhero[] team = new Superhero[3];  
team[0] = new WonderWoman();  
team[1] = new Batman();  
team[2] = new Stargirl();  
  
for (Superhero hero: team) {  
    hero.showCourage();  
}
```

```
class Director {  
    public Superhero[] assembleTeam() {  
        Superhero[] team = new Superhero[3];  
        team[0] = new WonderWoman();  
        team[1] = new GreenLantern();  
        team[2] = new Stargirl();  
        return team;  
    }  
}
```

Director d = new Director();
Superhero[] team = d.assembleTeam();
for (Superhero hero: team) {
 hero.showCourage();
}

Aha! Mit Polymorphie kann ich auch hier problemlos neue Unterklassen einführen.



Jetzt Seite 191 wiederholen!



Sie haben dafür 2 Minuten Zeit.

! ? Quiz

🎨 Polymorphie

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Bei Polymorphie können Referenz- und Objekttyp unterschiedlich sein, wenn ein Objekt deklariert und zugewiesen wird.

Richtig!

Bei Polymorphie kann der Referenztyp eine Unterklasse des tatsächlichen Objekttyps sein.

Eine Oberklasse!

In Java ist es auch möglich, polymorphe Argumente und Rückgabetypen zu verwenden.

Richtig!

Einer der Vorteile der Polymorphie ist, dass Sie Code schreiben können, der nicht geändert werden muss, wenn Sie das Programm um neue Unterklassen erweitern.

Richtig!

Von einer Klasse, die weder als `public` noch als `private` gekennzeichnet ist, können Unterklassen nur von Klassen abgeleitet werden, die sich im gleichen Package wie die Klasse selbst befinden.

Richtig!

Eine als `final` gekennzeichnete Klasse bedeutet, dass sie das Ende der Vererbungslinie darstellt. Eine finale Klasse kann nicht erweitert werden.

Richtig!

Wenn Sie eine bestimmte Methode vor dem Überschreiben schützen wollen, markieren Sie die Methode mit dem Modifier `final`.

Richtig!

 Welcher **eine** Gedanke zu
„ Polymorphie“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

Überschreiben und Überladen

**In der objektorientierten Programmierung
zwei ähnliche  und doch
unterschiedliche  Konzepte.**

Jetzt Seiten 192



und 193



wiederholen!



Sie haben dafür 5 Minuten Zeit.

! ? Quiz

🔧 Überschreiben und Überladen

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Eine Klasse `HomeAppliance` (Haushaltsgerät) definiert die booleschen Methoden `turnOn()` und `turnOff()`. Eine Unterklasse `MicrowaveOven` (Mikrowellenherd) definiert die boolesche Methode `turnOn(int cookingPower)`. Man sagt dann: diese Methode überschreibt die gleichnamige Methode der Superklasse.

Nein, dies ist eine Überladung, kein Überschreiben.

Der Compiler stellt sicher, dass eine Methode, die über eine Referenz aufgerufen wird, im Referenztyp deklariert ist. Zur Laufzeit betrachtet die JVM nicht den Referenztyp, sondern das tatsächliche Objekt auf dem Heap.

Richtig!

Beim Überschreiben einer Methode müssen die Argumente gleich sein und die Rückgabetypen müssen kompatibel sein. Letzteres bedeutet, dass die überschreibende Methode den gleichen Rückgabotyp oder einen Unterklassentyp davon deklarieren muss.

Richtig!

Eine Methode zu überladen bedeutet nichts anderes, als zwei Methoden zu definieren, die den gleichen Namen haben, aber unterschiedliche Argumentlisten verwenden.

Richtig!

Auch wenn Sie eine Methode überladen,
müssen die Rückgabetypen immer
kompatibel sein.

Solange die Argumentlisten unterschiedlich sind, können Sie die Rückgabetypen beliebig ändern.

 Welcher **eine** Gedanke zu
„ Überschreiben und
Überladen“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 Polymorphe Arrays

🤝 Pair Programming

- 🎯 Erstellen Sie eine Hierarchie mit der Superklasse `Animal` und den direkten Unterklassen `Hippo`, `Lion` und `Wolf`.
- ✅ Jedes Tier soll ein charakteristisches Geräusch auf der Konsole ausgeben. Implementieren Sie dazu die Methode `void makeNoise()`.



Polymorphe Arrays

Pair Programming

 Erstellen Sie eine Hierarchie mit der Superklasse `Animal` und den direkten Unterklassen `Hippo`, `Lion` und `Wolf`.

 Jedes Tier soll ein charakteristisches Geräusch auf der Konsole ausgeben. Implementieren Sie dazu die Methode `void makeNoise()`.

 Erstellen Sie ein Array namens `zoo`, in das Sie eine Reihe konkreter Tiere eintragen.

 Iterieren Sie über das Array `zoo` und lassen Sie die einzelnen Tiere ihr jeweiliges Geräusch ausgeben.

 Auf der Konsole werden die charakteristischen Tiergeräusche in der Reihenfolge des Arrays angezeigt.


```
1 import java.util.*;
2
3 public class PolymorphicArrayTestDrive {
4     public static void main(String[] args) {
5
6         // Note: We can assign values to an array during initialization.
7         // https://dev.java/learn/language-basics/arrays/#creating-initializing-accessing
8         // This shortcut syntax is also referred to as array literals:
9         Animal[] zoo = new Animal[] {
10             new Lion(),
11             new Wolf(),
12             new Wolf(),
13             new Wolf(),
14             new Hippo(),
15             new Hippo()
16         };
17
18         for (Animal resident : zoo) {
19             resident.makeNoise();
20         }
21
22     }
23 }
24
```

STDIN

Input for the program (Optional)

Output:

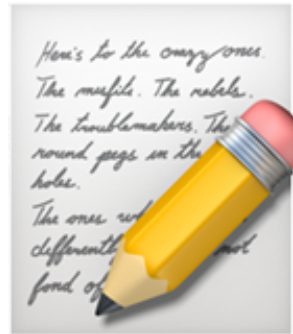
Roar!
Howl!
Howl!
Howl!
Growl!
Growl!



Welcher **eine** Gedanke zu



„Polymorphe Arrays“ war
für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren

 **Zielgerade**

↔ Teach Back

📖 Vererbung und Polymorphie

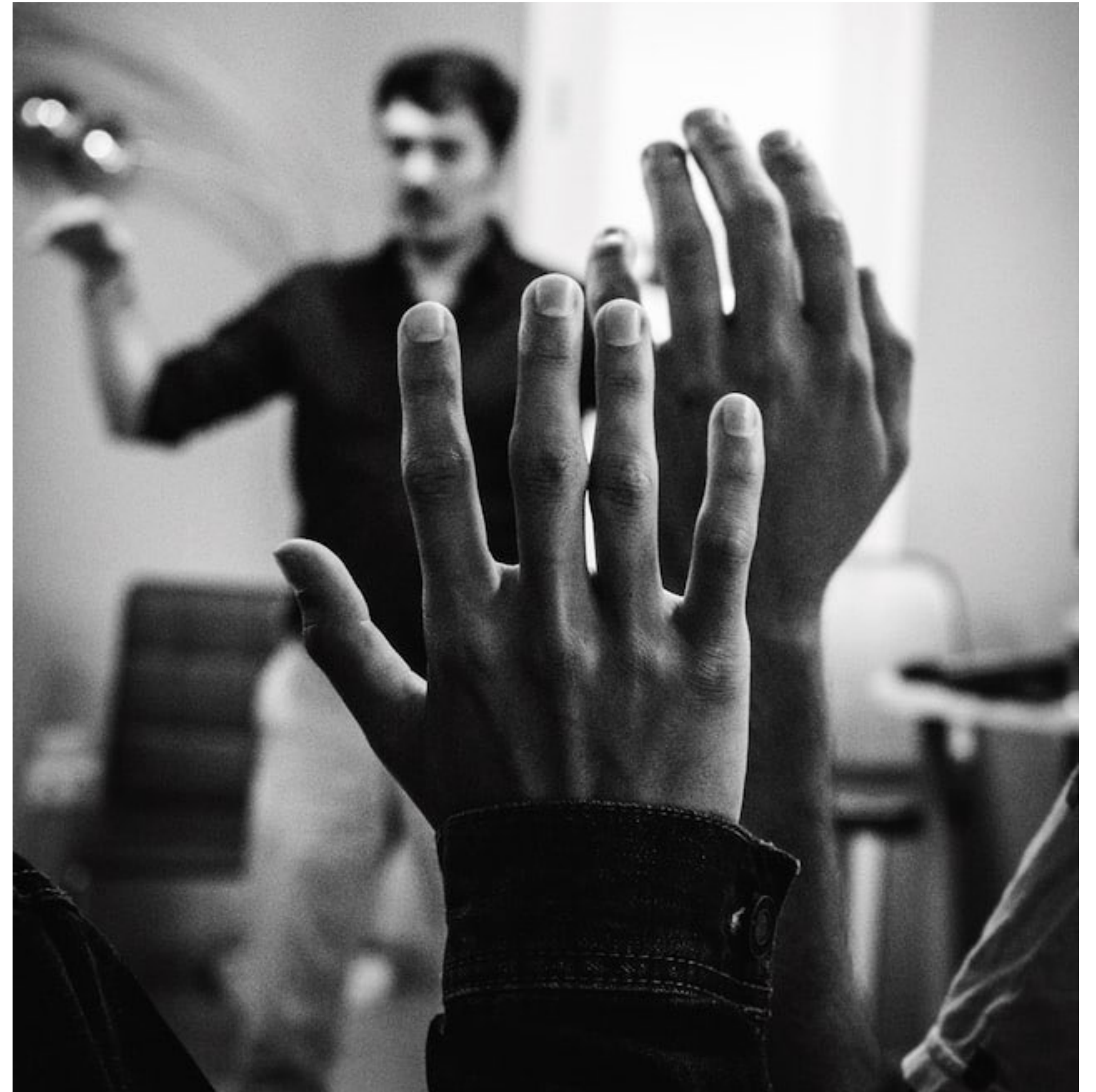
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Vererbung und Polymorphie**?
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Vererbung und Polymorphie



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

